CS135 HW4

yunzhe.li

October 2026

1a. Neural network diagram

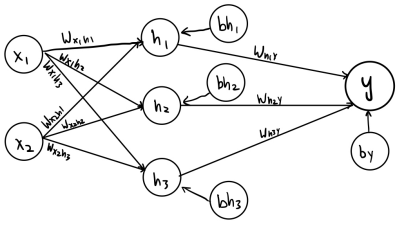


Figure 1: Neural network diagram

1b. Neural Network Dimensions

Component	Description	Shape	Total Parameters
Input X	Flattened Image Vector	784×1	NA
w_1	Input to h_1 weights	784×128	100,352
b_1	h_1 bias	128×1	128
w_2	h_1 to h_2 weights	64×128	8,192
b_2	h_2 bias	64×1	64
w_3	h_2 to Output weights	10×64	640
b_3	Output bias	10×1	10
Total	**Entire Network**	**NA**	109,386

Figure 2: Network dimensions

2.

The weight that could cause this [1, -1, -1, 1]and the hidden biases are [3, 1, -1, -3]. The input weights are 1. With output bias 0, the function is

$$y = \text{ReLU}(x + 3) - \text{ReLU}(x + 1) - \text{ReLU}(x - 1) + \text{ReLU}(x - 3).$$

Its slope changes at $x = -3, -1, 1, 3$, yielding zero outside the hat, height 2 between -1 and 1, and sloped sides.

3a.

Node	Value
u_1	1.5
v_1	1.5
u_2	-1.5
v_2	0
u_3	3.5
E	12.25

Table 1:

3b.

Node	Value
$\frac{dE}{du_1}$	21
$\frac{dE}{dv_1}$	21
$\frac{dE}{du_2}$	0
$\frac{dE}{dv_2}$	21
$\frac{dE}{du_3}$	7

Table 2:

4a.

Derivative of Sigmoid: $\frac{1}{1+e^{-x}}$
Max value: $\frac{1}{4} = 0.25$ at $u = 0$

4b.

Using the derivative expression given in the prompt, the four sigmoid-derivative factors are each at most 1/4, and the weights are 1. Therefore,

$$\left|\frac{\partial L}{\partial w_1}\right| \leq |y_1 - \hat{y}_1|\,|x_1| \left(\frac{1}{4}\right)^4 = \boxed{\frac{|y_1 - \hat{y}_1|\,|x_1|}{256}}.$$

With the prompt's signed formula, the corresponding bound is $(y_1 - \hat{y}_1)x_1/256$; the sign depends on $y_1 - \hat{y}_1$ and x_1.

4c.

Backpropagation multiplies the sigmoid derivatives across layers. Each is at most 1/4, so in a four-layer chain their product is at most 1/256 (with weights equal to 1). In deeper networks, these products can shrink rapidly, making gradients for early layers very small. This is the vanishing-gradient problem.

5a.

If $w^T x + b \leq 0$ for every training example, the node outputs

$$f(w^T x + b) = 0$$

on every example. Its local gradient is

$$f'(w^T x + b) = 0$$

under the convention in the prompt, including when the pre-activation equals zero.

5b.

The node receives zero gradient, so its weights and bias get no updates from those examples. The zero local gradient also blocks the gradient flowing to earlier layers through this node. The node can remain inactive during training—often called a "dead ReLU." Possible ways out include changing the initialization so the node activates on some training examples, or using an activation with a nonzero gradient for negative inputs, such as Leaky ReLU.

6a.

Approach 1 (the red, noisier path) probably used the smaller batch size. A smaller batch gives a less stable estimate of the full-data gradient, so updates vary more between steps. Approach 2 probably used the larger batch size and follows a smoother path.

6b.

Approach 1 is more likely to jitter around the minimum. Its smaller batch produces noisier gradient estimates, so its updates continue to vary even near the minimum.

6c.

To make Approach 1's steps look smaller and steadier, lower its learning rate. To make Approach 2's steps look larger, raise its learning rate. The learning rate changes the size of each update; it can change the path's scale, though it does not remove or add the underlying batch-gradient noise.

6d.

A batch that is too small can produce noisy updates, unstable progress, and poor use of parallel hardware. A batch that is too large requires more memory and computation per update, and it provides fewer updates for the same number of examples processed. Its gradients are more stable, but the reduced noise can make optimization less able to move away from some difficult regions.